

Custom Network Protocol Toolkit

Build log · visual explanations · interview preparation

A binary protocol from scratch · threaded and async servers
reliable UDP · packet crafting · benchmarks

Parth Hirpara · github.com/theHparth/netproto

How to use this guide

If you want to...	Go to
See how the files fit together	Part A — the big picture
Understand something you got stuck on	Part B — concepts with pictures
Rebuild the project from zero	Part C — step-by-step commands
Revisit a specific confusion	Part D — the confusion log
Practise for the interview	Part E — question bank
Cram the night before	Part F — one-line answers
Look up a word	Part G — glossary

REAL LIFE

Orange boxes are **real-life examples** — read these first if a section is confusing.

Green = **key point**. Amber = **a trap**. Blue = **the exact words to say in the interview**.

The 30-second summary of the whole project

TCP hands you a stream of bytes with no message boundaries. I designed an 18-byte binary header that says how long each message is, plus a checksum. Then I built two servers that speak it — one thread-per-client, one asyncio — measured both, rebuilt reliable delivery on top of UDP by hand, and wrote a Scapy tool that crafts and dissects my own packets.

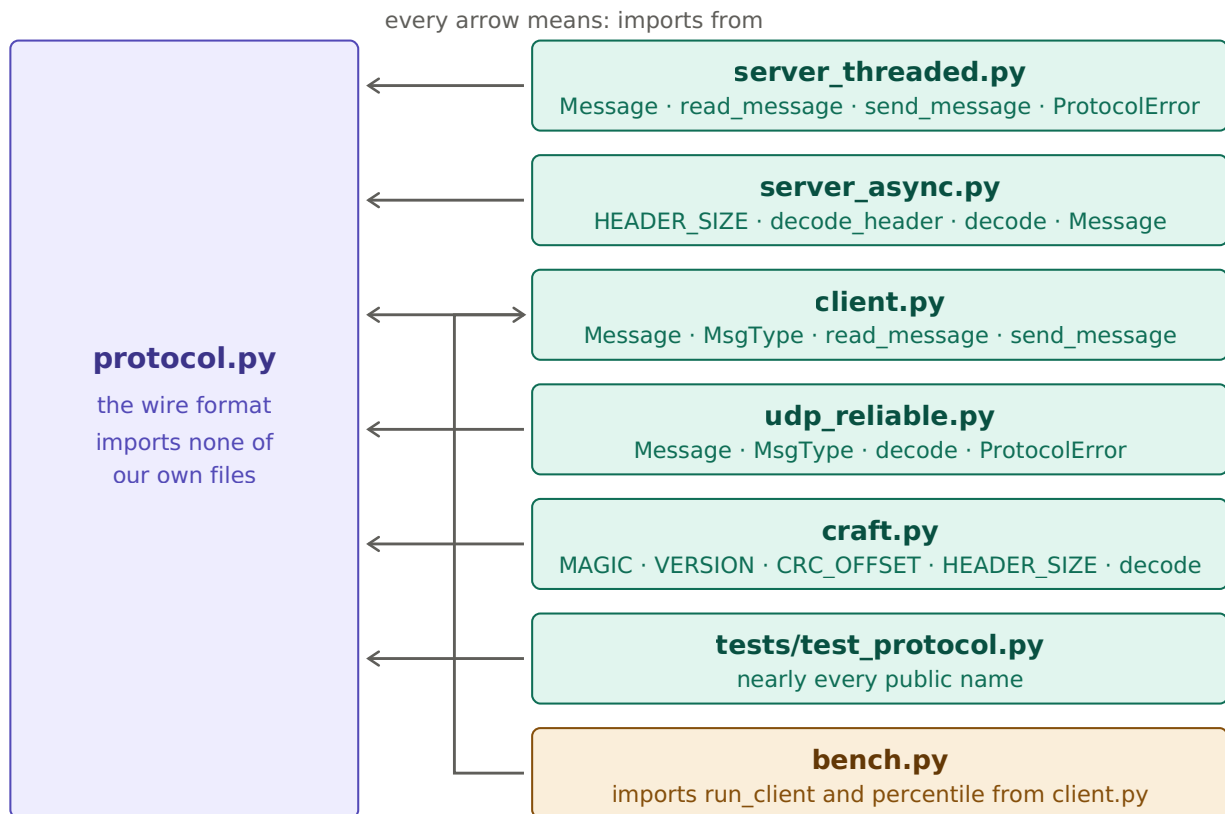
Part A · The big picture

A1 · What actually got built

File	Job
<code>protocol.py</code>	The wire format. Message → bytes and back. Everything imports it.
<code>server_threaded.py</code>	TCP server, one OS thread per client.
<code>server_async.py</code>	Same server, one thread with an event loop.
<code>client.py</code>	Sends messages, verifies every reply, measures speed.
<code>udp_reliable.py</code>	Reliable delivery over UDP: sequence numbers, ACKs, retries.
<code>craft.py</code>	Scapy: build raw packets, save a pcap, decode it with our own parser.
<code>bench.py</code>	Runs both servers at 10/100/500 clients, prints a results table.
<code>tests/test_protocol.py</code>	44 pytest cases.

A2 · Which file uses which — the dependency map

This is the question that trips everyone up: *a function from one file is used in another file — how do I keep track?* Here is the whole answer in one picture.



Everything points at protocol.py. Only bench.py points somewhere else.

REAL LIFE

`protocol.py` is the **dictionary**. The servers and clients are the **people having a conversation**. Everyone consults the same dictionary, so everyone agrees what the words mean. Nobody writes their own.

A3 · The function map — where every name lives

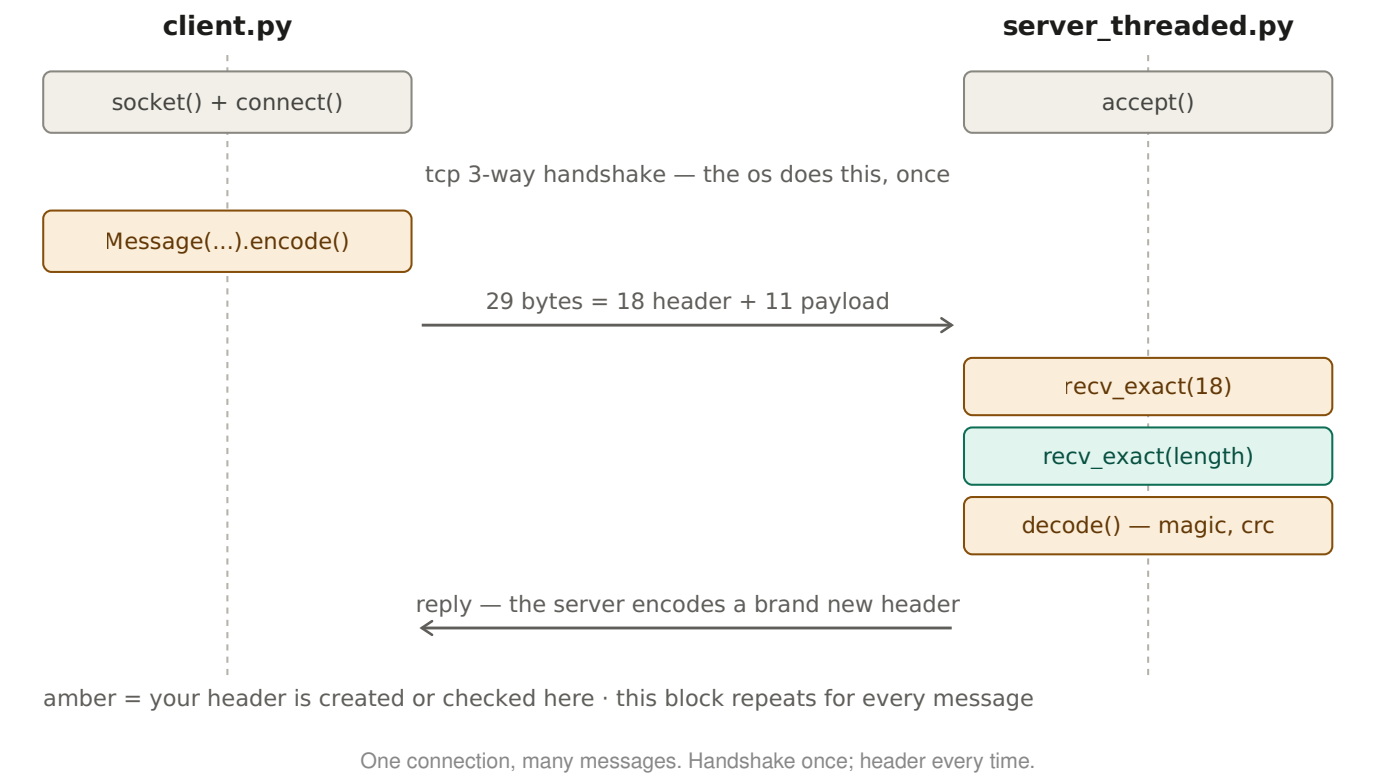
Name in <code>protocol.py</code>	What it does	Used by
<code>Message</code>	holds type, seq, payload	every file
<code>MsgType</code>	names for the type byte	every file
<code>.encode()</code>	object → bytes	servers, client, udp, craft
<code>decode_header()</code>	18 bytes → validated header fields	<code>read_message</code> , <code>server_async</code>
<code>decode()</code>	whole frame → <code>Message</code>	<code>read_message</code> , udp, craft, tests
<code>recv_exact()</code>	read exactly N bytes off a socket	<code>read_message</code> , tests
<code>read_message()</code>	socket → <code>Message</code> (the two-read dance)	<code>server_threaded</code> , client
<code>send_message()</code>	<code>Message</code> → socket	<code>server_threaded</code> , client
<code>ProtocolError</code> + 5 children	one family of failures	servers, udp, craft, tests

Name elsewhere	Defined in	Used by
<code>run_client()</code>	<code>client.py</code>	<code>bench.py</code>
<code>percentile()</code>	<code>client.py</code>	<code>bench.py</code>
NPTK (Scapy layer)	<code>craft.py</code>	<code>craft.py</code> only

KEY POINT

When you get lost, ask one question: **where is this name defined?** In PyCharm, `Ctrl+B` or `Ctrl+click` on any name jumps straight to it.

A4 · The life of one message, end to end



Happens once per connection	Happens once per message
connect() / accept()	encode() — header created
TCP 3-way handshake	sendall()
close()	recv_exact() twice
	decode() — header checked

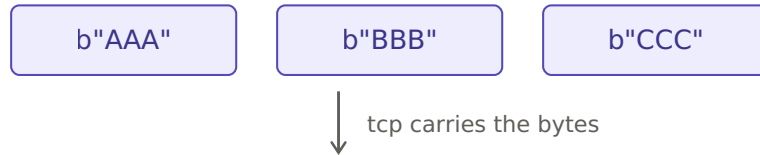
TRAP

"A new connection is made for every message." **No.** One connection carries thousands of messages. That is exactly why seq exists — to tell message 1 from message 2 inside the same connection.

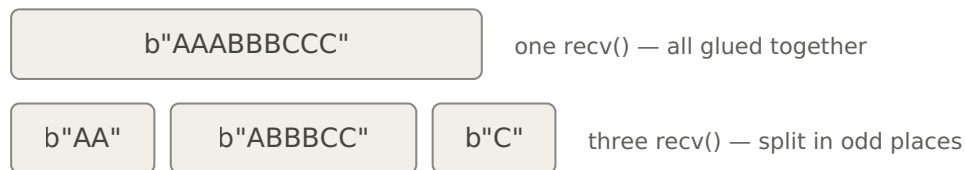
Part B · The concepts, with pictures

B1 · Why TCP needs a header at all

you call `send()` three times



the other side may receive any of these



tcp promises order and no corruption. it never promises where a message ends.

The problem every framing layer exists to solve.

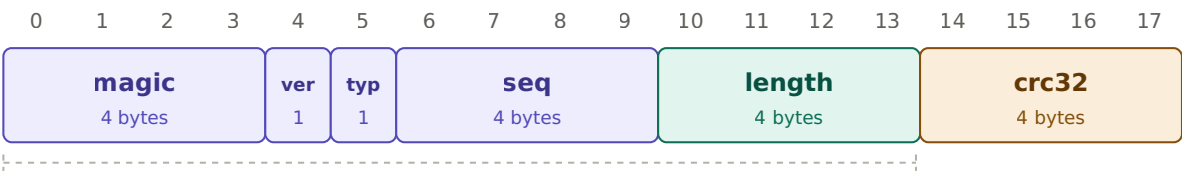
REAL LIFE

You hand three letters to a post office. It delivers one long ribbon of paper with all your words printed end to end — right order, nothing missing, **no envelopes**. You cannot tell where one letter stops.

The fix: write at the top of each letter **"this letter is 300 characters long"**. That line is the header. That is **length-prefix framing**.

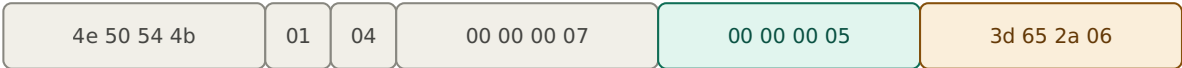
Option	How	Problem
Delimiter	end each message with <code>\n</code>	the marker cannot appear in your data → you must escape it → size becomes unpredictable
Length-prefix ✓	put the size in a fixed header up front	receiver must do two reads instead of one

B2 · The header we designed



crc is computed over these 14 bytes plus the payload

example: an echo frame carrying b"hello"



then exactly LENGTH payload bytes follow



total on the wire: 18 + 5 = 23 bytes

The 18-byte header, then the payload.

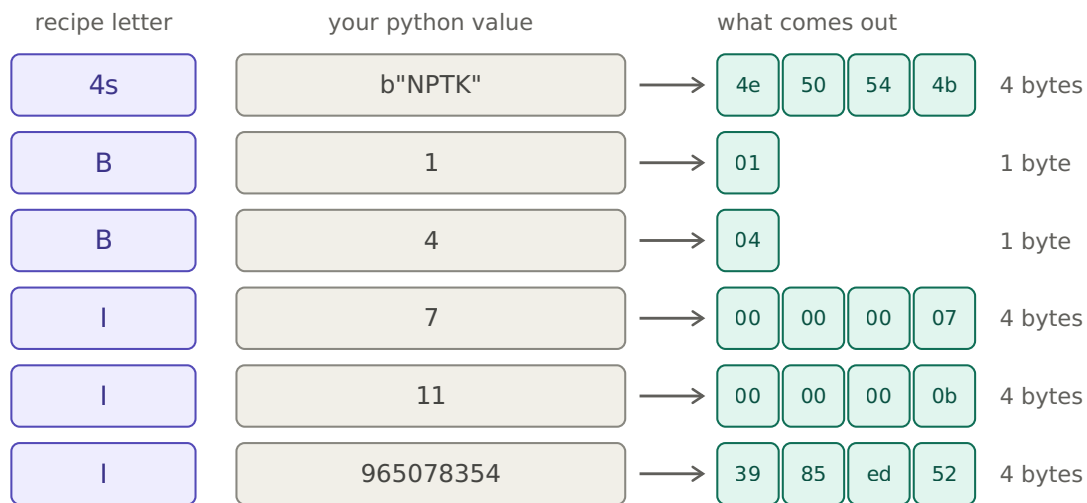
Field	Size	Why it exists	Without it
MAGIC	4 B	four fixed letters NPTK	you would happily parse a browser's traffic as if it were yours
VERSION	1 B	"this is format v1"	a future v2 gets silently misread instead of rejected
TYPE	1 B	DATA / ECHO / ACK / GOODBYE	you have data but no idea what to do with it
SEQ	4 B	message number 1, 2, 3...	you cannot tell a duplicate from a new message
LENGTH	4 B	the actual fix — how many bytes follow	you cannot find the end of the message
CRC32	4 B	a fingerprint of the contents	a flipped bit slips through unnoticed

REAL LIFE

Magic numbers are real. PNG files start with .PNG , PDFs with %PDF , ZIPs with PK. . , Java class files with CAFEBAFE . That is how your computer knows a file is a PNG even if you rename it to cat .txt .

B3 · struct.pack — the label printer

HEADER_FORMAT = "!4sBBIII" is a **recipe string**. Each letter eats exactly one Python value and prints a fixed number of bytes.



6 python values in → 4 + 1 + 1 + 4 + 4 + 4 = 18 bytes out

struct.pack, one letter at a time.

Letter	Name	Size	Max value
s	string of raw bytes	as declared	—
B	Byte	1 byte	255
I	Int	4 bytes	4,294,967,295
!	big-endian, no padding	—	—

What the ! fixes

```
struct.pack("!I", 7)    -> b'\x00\x00\x00\x07'    # 00 00 00 07
struct.pack("<I", 7)    -> b'\x07\x00\x00\x00'    # 07 00 00 00
```

Order	The number 7 as 4 bytes	Used by
! big-endian	00 00 00 07	the internet
< little-endian	07 00 00 00	Intel chips (your laptop)

TRAP

If the sender writes big-endian and the reader assumes little-endian, **7 arrives as 117,440,512**. The ! makes everyone agree. Its nickname is **network byte order**.

B4 · `encode()` — object to bytes

step 1 — you create the object

```
Message(ECHO, seq=7, payload=b"hello world")
```



step 2 — pack the first 14 header bytes, crc left out

```
4e 50 54 4b 01 04 00 00 00 07 00 00 00 0b
```



step 3 — `zlib.crc32(prefix + payload)`

```
39 85 ed 52
```



step 4 — glue in order: $14 + 4 + 11 = 29$ bytes

```
14-byte prefix
```

```
crc
```

```
hello world
```

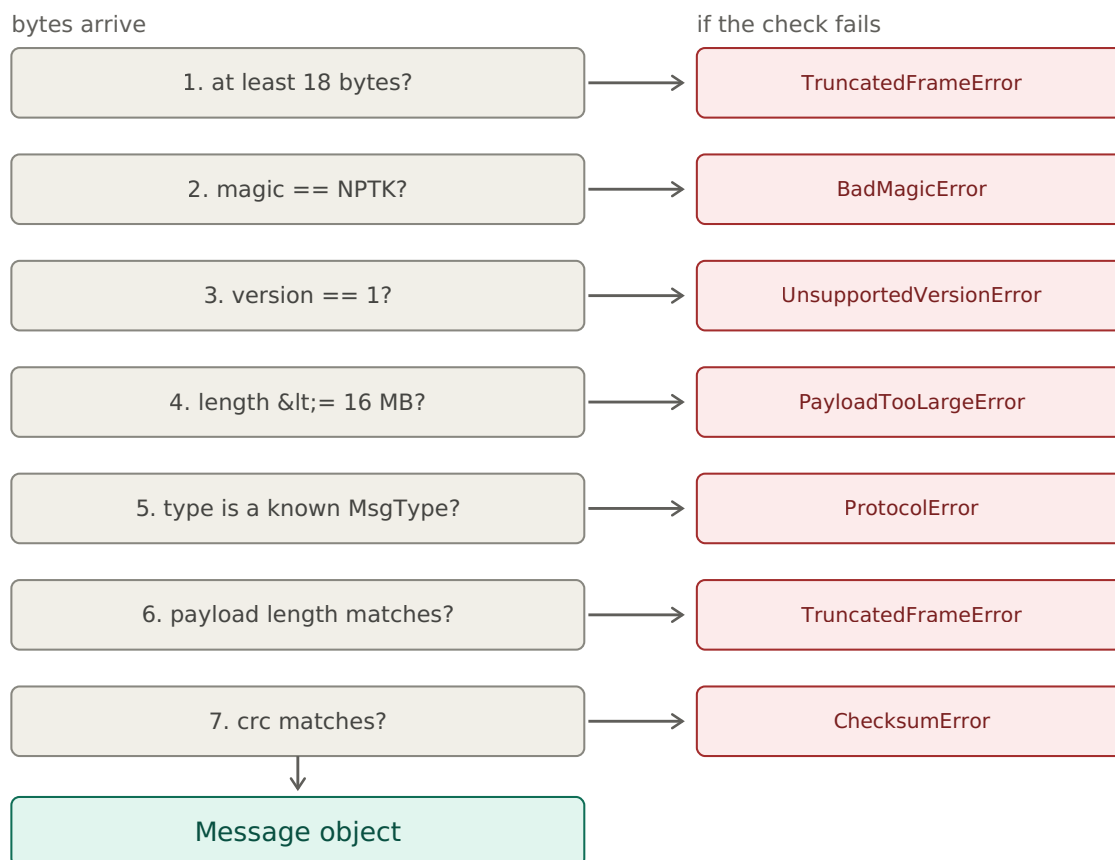
`encode()`: object in, 29 bytes out.

KEY POINT

The **order** is the trick. You cannot pack all 18 bytes first, because you do not know the CRC yet. So: pack 14 → checksum those 14 plus the payload → append the 4-byte result → append the payload.

REAL LIFE

A fingerprint cannot include itself. That is why the CRC covers bytes 0–13 and stops — chicken and egg.



Cheapest checks first. The CRC touches every byte, so it runs last.

Rule	Why it matters
Cheap checks before expensive ones	someone flooding you with bad-magic frames should not cost you a CRC computation each time
Size cap before allocating	this single line is the difference between a bug and a denial-of-service vulnerability

TRAP

`LENGTH` is a 4-byte unsigned int, so a peer can legally declare **4,294,967,295**. A naive server does `bytearray(length)` — four gigabytes allocated from one 18-byte packet. Cap it **before** you allocate.

B6 · `recv_exact()` — the bug that survives testing

you want 18 bytes for the header. one `recv()` is not enough.

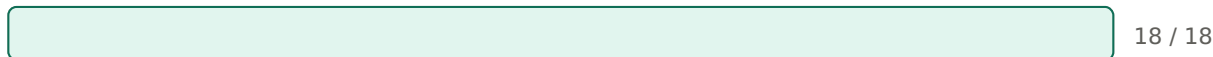
1st call: `recv(18)` → returned 7



2nd call: `recv(11)` → returned 6



3rd call: `recv(5)` → returned 5



only now can you unpack the header

`recv()` returns UP TO `n` bytes — never assume you got `n`.

What you think <code>recv(18)</code> means	What it actually means
return 18 bytes	return up to 18 bytes — maybe fewer
block until 18 arrive	block until at least 1 arrives, then return

```
def recv_exact(sock, n):
    if n == 0:
        return b""                # recv(0) returns b"" -> looks like a hang-up
    buf = bytearray()
    while len(buf) < n:
        chunk = sock.recv(n - len(buf)) # ask only for what is MISSING
        if not chunk:                  # b"" means the peer closed
            raise TruncatedFrameError(...)
        buf.extend(chunk)
    return bytes(buf)
```

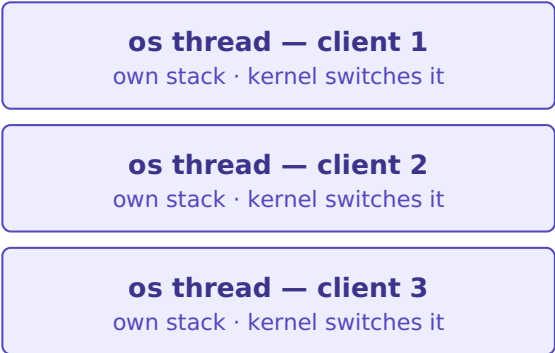
Line	Why exactly this
<code>bytearray()</code> not <code>b""</code>	bytes are immutable — <code>buf += chunk</code> copies everything each time, $O(n^2)$
<code>n - len(buf)</code>	asking for <code>n</code> again could swallow the start of the next message and desync the stream forever
<code>if not chunk</code>	without it, a closed connection spins the loop at 100% CPU forever
<code>if n == 0</code>	a zero-length payload (GOODBYE, ACK) would otherwise look like a dead connection

TRAP

This bug **passes every test on localhost**, because there is no real fragmentation. It only appears under real latency, larger payloads or high concurrency. That is exactly why it is an interview question.

B7 · Threads vs asyncio

threaded



kernel decides who runs · ~8 MB stack each

asyncio



tasks yield only at await · ~KB each

Same protocol, two ways to be concurrent.

REAL LIFE

Restaurant. Threaded = 10 waiters, one per table. Asyncio = 1 waiter for 10 tables. The single waiter copes because tables spend most of their time *thinking*, not talking. It breaks the moment one table asks the waiter to **go and cook** — that is a blocking call freezing the whole event loop.

	Threaded	Asyncio
Cost per connection	~8 MB stack	a few KB
Who switches	the kernel, anywhere	you, only at <code>await</code>
Needs locks for shared state	yes	usually no
Practical ceiling	low thousands (C10K)	tens of thousands
Killed by	too many threads	one blocking call

KEY POINT

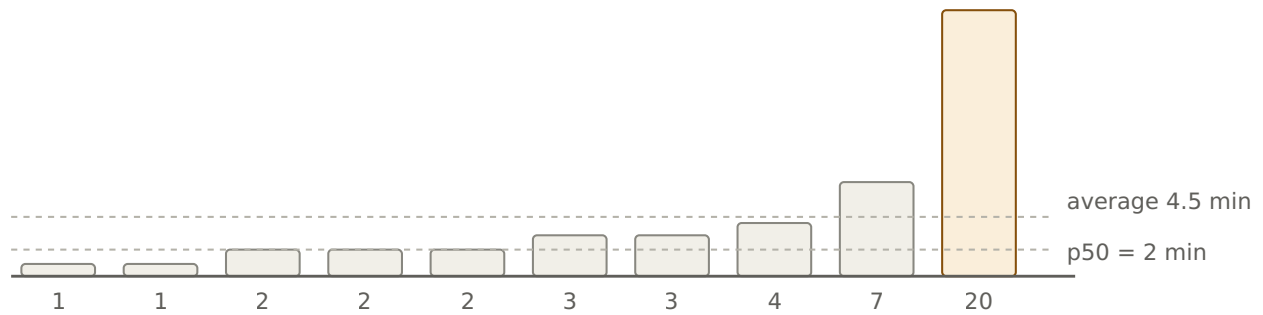
`counter += 1` is **not** thread-safe in Python — it is load, add, store, and a thread can be interrupted between them. In asyncio it *is* safe, because a coroutine can only be suspended at an `await`.

B8 · Latency, throughput and percentiles

REAL LIFE

Percentile is exam ranking. "I got 95 percentile" does not mean you scored 95 — it means **95% of students scored less than you**. Same idea here: line up every request from fastest to slowest, walk to the 95th, read your stopwatch.

10 customers, waiting time in minutes, sorted



not one customer waited 4.5 minutes — the average was invented by the slow one

Why the mean is not a real request.

Metric	Reads as	Who feels it
p50 (median)	half the requests were faster than this	the typical user
p95	95 out of 100 were faster	1 user in 20
p99	99 out of 100 were faster	1 user in 100
mean	sum ÷ count	nobody — it is not a real request

Why the tail matters more than it sounds

A web page loads 20 things — images, CSS, fonts, API calls. If 5% of requests are slow (that is what p95 means), what is the chance the *page* feels slow?

Requests on the page	Chance all are fast	Chance at least one is slow
1	95%	5%
5	77%	23%
10	60%	40%
20	36%	64%

$0.95^{20} = 0.36$. Two out of three page loads contain a slow request. Users experience the **slowest thing on the page**, not the median.

Latency is not throughput

Question	Name	Measured in
"how long did I wait?"	latency	milliseconds per request
"how many customers did the shop serve?"	throughput	requests per second

REAL LIFE

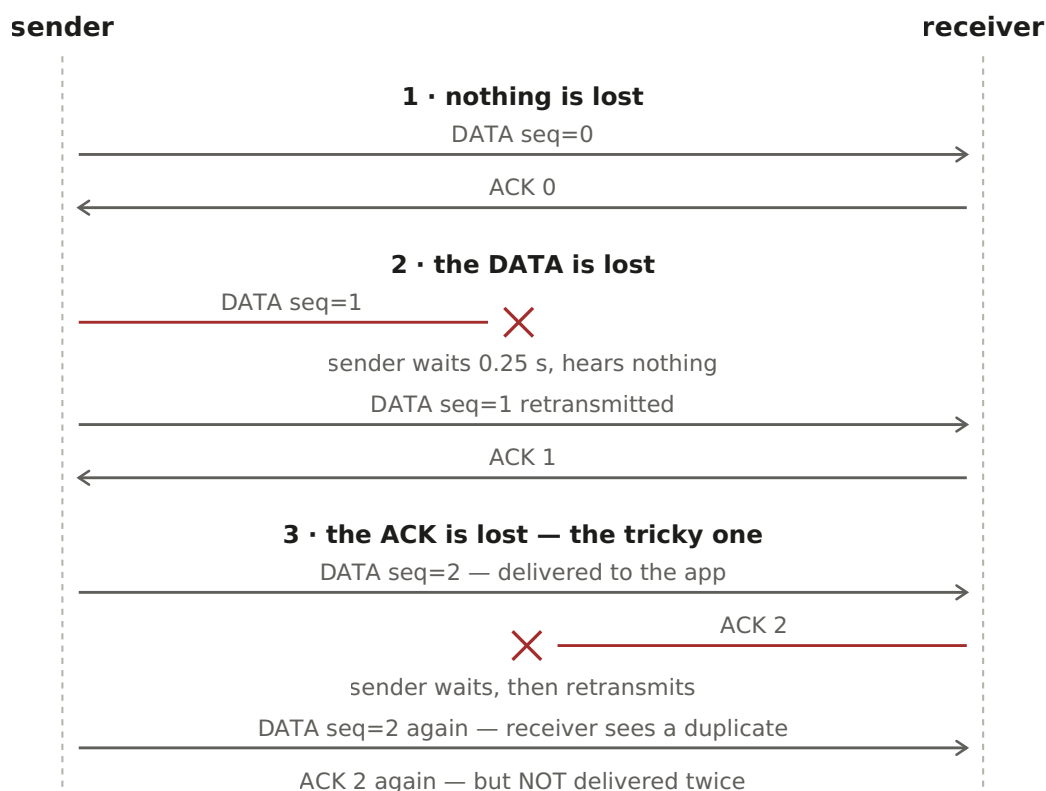
Elevator. A huge slow elevator moves many people per hour (high throughput) but you wait ages for it (high latency). A small fast one arrives instantly (low latency) but moves fewer people (low throughput). Neither is "faster".

B9 · Reliable UDP — stop-and-wait ARQ

	TCP	UDP
Delivery guaranteed	yes	no
Order preserved	yes	no
Duplicates removed	yes	no
Speed / overhead	connection setup, slower	fast, no setup

REAL LIFE

UDP is a **postcard** — cheap, might not arrive, nobody tells you. TCP is **registered post** — costs more, you get a delivery receipt. `udp_reliable.py` builds the receipt system by hand.



The sender cannot tell case 2 from case 3 — both look like silence.

KEY POINT

The sender cannot tell case 2 from case 3. A lost DATA and a lost ACK both look like *silence*. So it always retransmits — which means the receiver **will** see duplicates. That is the entire reason `seq` exists.

Receiver rule	Why
Deliver only if <code>seq == expected</code>	otherwise the app sees the same message twice
Always send an ACK, even for a duplicate	the only reason it arrived twice is that your last ACK was lost. Silence = the sender retries forever.
Never ACK a datagram that failed the CRC	the sender's timer fires and retransmits — corruption and loss share one recovery path

Measured cost of reliability

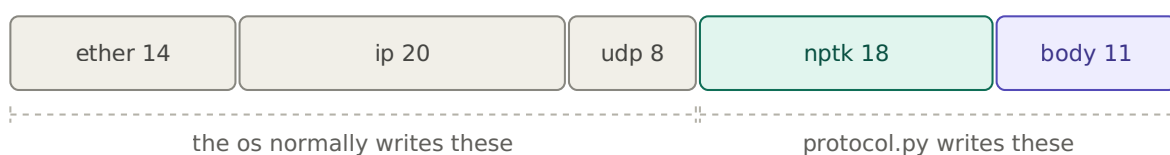
loss	messages	transmissions	efficiency	delivered exactly once
0%	6	6	100%	yes
20%	5	7	71%	yes
30%	6	19	32%	yes

TRAP

Stop-and-wait sends one message per round trip. On loopback that is 10,000/s. On a 200 ms link it is **5 per second**, no matter how fat the pipe. The fix is a **sliding window** — stop-and-wait is simply window size 1.

B10 · Crafting packets with Scapy

one crafted packet — 71 bytes on the wire



with scapy you write all five by hand — that is the point of craft.py

Your header sits on top of three the OS usually fills in.

REAL LIFE

Normally the post office prints the envelope for you. Scapy lets you **write the envelope by hand** — including the return address. That is powerful (you can test malformed input) and it is also how spoofing attacks work.

Mode	Needs admin?	What it does
<code>--mode build</code>	no	build one packet, print every field, compare with protocol.py
<code>--mode pcap</code>	no	build → write capture.pcap → read back → decode
<code>--mode sniff</code>	yes	live capture off a network interface
<code>--mode send</code>	yes	actually transmit raw packets

KEY POINT

The payoff line from `--mode build` is `scapy bytes == protocol.py bytes : True`. Two independent implementations of the same spec produced identical bytes. That is what having a real spec means.

Part C · Rebuild it from zero

Every command, in order. Commit after each step.

Step 0 · Environment and repository

```
python --version          # need 3.10+
git --version

git config --global user.name "Parth"
git config --global user.email "usphirpara@gmail.com"

mkdir netproto && cd netproto && mkdir tests
python -m venv .venv
.venv\Scripts\activate     # windows
source .venv/bin/activate  # mac / linux
pip install -r requirements.txt

git init && git add . && git commit -m "chore: project skeleton"
git branch -M main
git remote add origin https://github.com/theHparth/netproto.git
git push -u origin main
```

File	Why it exists
<code>.gitignore</code>	keeps <code>.venv/</code> and <code>__pycache__</code> out of the repo — a committed venv is an instant red flag
<code>requirements.txt</code>	anyone can recreate your environment with one command
<code>README.md</code>	written up front so it is not an afterthought

TRAP

The repo must be created on github.com **first**. `git push` never creates one for you — that is what "Repository not found" means. Also: do not keep a git repo inside OneDrive; it tries to sync every file in `.git/`.

Step 1 · `protocol.py`, in four passes

Pass	What goes in
1.1	constants, <code>MsgType</code> , the five exceptions
1.2	<code>Message</code> dataclass + <code>encode()</code>
1.3	<code>decode_header()</code> + <code>decode()</code>
1.4	<code>recv_exact()</code> , <code>read_message()</code> , <code>send_message()</code>

```
python protocol.py          # runs the self-check block
# -> HEADER_SIZE : 18 bytes, CRC_OFFSET : 14, self-check OK
```

KEY POINT

`if __name__ == "__main__":` means **run this only when I am the file being executed, not when someone imports me**. It is how one file can be both a library and a script.

Step 2 · `server_threaded.py`

```
python server_threaded.py          # terminal 1
# threaded server listening on 127.0.0.1:9000 (backlog 128)
```

Socket call	Plain English	Skip it and...
<code>socket()</code>	give me a TCP/IPv4 socket	nothing to work with
<code>setsockopt(SO_REUSEADDR)</code>	let me rebind this port immediately	"Address already in use" for ~60 s after every restart
<code>bind()</code>	claim 127.0.0.1:9000	OS gives you a random port
<code>listen(128)</code>	start queueing incoming connections	<code>accept()</code> fails
<code>accept()</code>	block until someone connects — returns a new socket	—

TRAP

`listen(128)` is **not** a limit of 128 clients. It is 128 clients *waiting to be accepted*. You can serve 10,000 with `listen(128)`.

KEY POINT

`TCP_NODELAY` turns off Nagle's algorithm. Nagle batches small writes, which adds up to **40 ms** per reply. For a request/response protocol that is pure loss.

Step 3 · `client.py`

```
python client.py --count 3000 --size 64

throughput      : 10,988 msg/s   (1.7 MiB/s both directions)
latency p50     : 0.075 ms
latency p95     : 0.137 ms
latency p99     : 0.166 ms
```

Detail	Why a real benchmark needs it
20 warmup messages, unmeasured	TCP slow start and cold code paths happen once and never repeat — including them inflates p99 with noise
<code>time.perf_counter_ns()</code>	monotonic and nanosecond-resolution. <code>time.time()</code> can jump backwards and has ~15 ms resolution on Windows
verify every reply	a benchmark that does not check correctness measures how fast you can be wrong
count header bytes and both directions	reporting payload only would hide your framing overhead

Step 4 · `server_async.py`

```
python server_async.py          # port 9001
python client.py --port 9001 --count 3000 # the SAME client
```

KEY POINT

`client.py` needed **zero** changes to talk to a completely different server. Same protocol, same bytes. That is the payoff of keeping `protocol.py` transport-agnostic.

Threaded	Async	Note
<code>recv_exact(sock, n)</code> — our loop	<code>await reader.readexactly(n)</code>	asyncio already wrote the loop
<code>sock.sendall(...)</code>	<code>writer.write(...)</code> + <code>await writer.drain()</code>	<code>write()</code> only buffers; <code>drain()</code> is the backpressure
<code>threading.Lock()</code>	no lock	coroutines only switch at <code>await</code>

TRAP

Forgetting `await writer.drain()` is the classic asyncio bug: `write()` never blocks, so a slow client makes the server buffer replies in RAM until the process dies.

Step 5 · `tests/`

```
pytest          # 44 passed in 0.10s
pytest -v       # one line per test
pytest -k crc   # only tests with "crc" in the name
```

pytest feature	What it buys you
<code>assert</code>	pytest rewrites it to show both sides on failure
<code>pytest.raises(ChecksumError)</code>	fails if nothing is raised AND if the wrong error is raised
<code>@parametrize</code>	one function → many named tests. A loop would stop at the first failure and hide the rest
fixtures	setup with teardown attached — code after <code>yield</code> runs even if the test fails
<code>pytest.ini</code> → <code>pythonpath = .</code>	without it <code>import protocol</code> fails inside <code>tests/</code>

Step 6 · `udp_reliable.py`

```
python udp_reliable.py --loss 0    --count 4
python udp_reliable.py --loss 0.2  --count 5 --seed 3
python udp_reliable.py --loss 0.3  --count 6 --seed 7
```

REAL LIFE

Why --loss ? A smoke alarm never beeps because there is never a fire — so you press the **TEST** button. On localhost UDP never drops packets, so the retransmit code would never run. Netflix does the same thing at scale and calls it **Chaos Monkey**.

REAL LIFE

Why --seed ? Minecraft world seeds. Same seed = same world, every time. Without it your bug is "the car makes a noise sometimes" and the mechanic can do nothing. With it, it is "turn left at 40 and you will hear it".

Step 7 · `bench.py`

```
python bench.py --quick          # ~20 s sanity check
python bench.py --clients 10 100 500 --count 100 # the real run
```

Detail	Why
<code>threading.Barrier</code>	a starter pistol. Without it the first clients finish before the last ones connect, so you never actually test 500 concurrency
poll the port instead of <code>sleep(2)</code>	flaky on a slow machine, wasteful on a fast one
<code>stdout=DEVNULL</code>	1,000 lines of logging would compete with the thing you are measuring
1 s pause between rounds	closed sockets sit in <code>TIME_WAIT</code> ; without the pause you run out of ephemeral ports

Step 8 · `craft.py`

```
python craft.py --mode build
python craft.py --mode pcap --count 3
# packet 3: FAIL ChecksumError <- the deliberately corrupted one
```

TRAP

On Windows, an empty `Ether()` makes Scapy look up your default network card's MAC — and it may pick a loopback adapter that does not exist. Fix: pass literal MACs. Deeper fix: realise you are writing to a **file**, so you never needed a network card at all.

Step 9 · README

Section	Why it earns points
Quick start	a stranger can actually run it — most GitHub projects fail this
ASCII header diagram + hex example	shows you designed a format, not copied one
Benchmark table	real numbers you measured
Benchmark caveats	the strongest signal in the whole repo — you know your own benchmark is not perfect
Design decisions	each one states the choice, the reason, and the cost
Known limitations	you know what your code cannot do

Part C2 · The benchmark results, and what they mean

□ threaded □ async



throughput in messages per second — taller is better

Async wins throughput at every level. Latency is a different story.

server	clients	msgs	throughput (msg/s)	p50 (ms)	p95 (ms)	p99 (ms)	max (ms)
threaded	10	1,000	10,303	0.581	2.398	3.145	6.983
threaded	100	10,000	10,518	4.078	13.473	19.741	40.184
threaded	500	50,000	11,641	5.878	16.940	25.358	60.875
async	10	1,000	14,439	0.494	1.143	3.270	3.858
async	100	10,000	14,147	5.509	13.572	19.193	42.435
async	500	50,000	17,162	23.427	40.885	52.835	93.443

TRAP

These are **my** numbers on Linux. Run `bench.py` on your own machine and replace them. You must be able to say "I watched this happen", not "a document told me".

The surprise: async won throughput but lost p99

At 500 clients	threaded	async	Winner
Throughput	11,641	17,162 (+47%)	async
p99 latency	25.4 ms	52.8 ms	threaded

async — one event loop, one queue



threaded — many workers, short queues



each person waits behind at most 1

one queue moves more people per hour — but the unlucky ones wait much longer

Throughput and tail latency are different axes.

#	Reason	Real-life version
1	Async runs on one CPU core . The threaded server's syscalls release the GIL, so the kernel overlaps 500 threads across all cores.	1 cashier vs 4 cashiers
2	The OS schedules per thread , so a 500-thread process gets a bigger slice of CPU than a 1-thread process.	4 employees get 4 lunch breaks; 1 gets 1
3	Head-of-line blocking : in an event loop one slow callback delays everyone behind it.	one customer arguing about a coupon holds up the line
4	The event loop is fair (FIFO) so everyone waits the same. Threads are unfair, so some finish fast.	one queue: all wait 20 min. Four queues: some 5, some 40

The maths that proves it is not a bug

Little's Law: $\text{average wait} = \text{concurrency} \div \text{throughput}$

Server	Concurrency	Throughput	Predicted wait	Measured p50
async	500	17,162 /s	$500 \div 17,162 = 29 \text{ ms}$	23 ms

KEY POINT

Throughput and tail latency are **different axes**. Higher throughput plus higher latency means a longer queue. That is a queue behaving exactly as theory predicts, not a broken benchmark.

SAY THIS OUT LOUD

"Async gave 47% more throughput at 500 connections but roughly double the p99 — because a single event loop serialises everything, while 500 OS threads let the kernel overlap syscalls across cores. Throughput and tail latency traded against each other, which is exactly what Little's Law predicts."

The follow-up they will ask

SAY THIS OUT LOUD

"So how would you make async win on latency too?"

"Run one event loop **per CPU core** instead of one total — separate processes binding the same port with `SO_REUSEPORT`, and the kernel load-balances between them. That is what nginx does, and what `gunicorn -w 4` does. You keep the low memory per connection and get real multi-core parallelism: four short queues instead of one long one."

Other things they may probe

They ask	You say
Is your benchmark fair?	Not fully — the load generator runs 500 threads on the same machine , so at 500 clients the client competes with the server for CPU. I would want a separate load box for real numbers.
Would async ever win on latency?	Yes — with many idle connections. 10,000 mostly-silent websockets would destroy the threaded server on memory; async would barely notice.
What if the work were CPU-heavy?	Both lose. The GIL means neither model gives real CPU parallelism in one process. You would need multiprocessing or a C extension.
Why does throughput flatten from 10 to 100 clients?	The server was already saturated at 10. Extra clients add queueing , not work done — which is why latency climbs while throughput does not.

Part D · The confusion log

Every question that came up while building this, with the plain answer and the version to say in an interview. These are the most valuable pages in the document — you only remember what you struggled with.

Setup and git

I asked	Plain answer	Interview version
What is a venv?	A private Python toolbox for one project. <code>pip install</code> goes inside it, not into system Python. Delete the folder and the project is clean again.	A venv per project plus a pinned <code>requirements.txt</code> gives a reproducible environment. Two projects can need different versions of the same library.
Where do the config files go?	Directly in <code>netproto/</code> , next to <code>.venv</code> and <code>tests</code> — not inside them.	—
Why <code>.gitignore</code> ?	It lists what git must never commit — <code>.venv/</code> , <code>__pycache__</code> , <code>.pytest_cache/</code> .	Anything generated or secret stays out of version control. If it can be regenerated from what IS committed, it does not belong.
What is <code><your-username></code> ?	Your GitHub username, not your Windows one. Find it at github.com under your avatar.	—
"Repository not found" on push	Either the remote URL is wrong, or the repo was never created on GitHub. Git never auto-creates repos.	—
<code>import struct</code> failed in the terminal	Wrong prompt. <code>C:\...></code> is Windows CMD; <code>>>></code> is Python. Type <code>python</code> to enter, <code>exit()</code> to leave.	—
Do we always use <code>>>></code> ?	No. The REPL is for throwaway experiments. The real project is <code>.py</code> files run with <code>python file.py</code> .	—

The protocol

I asked	Plain answer	Interview version
What is <code>NPTK</code> ?	A magic number we invented — 4 fixed bytes at the start of every frame. Like <code>%PDF</code> or <code>.PNG</code> .	Magic is a cheap sanity check. Wrong magic means the peer does not speak my protocol or I have lost stream sync — close rather than misparse bytes.
What is <code>"!4sBBIII"</code> ?	A recipe string for <code>struct</code> . Each letter eats one value: <code>4s</code> = 4 raw bytes, <code>B</code> = 1 byte, <code>I</code> = 4 bytes. Total 18.	<code>struct.pack</code> serialises, <code>struct.unpack</code> reverses it. One format constant so both directions can never drift apart.
What does <code>!</code> do?	Forces big-endian and no padding. <code>pack("!I", 7) → 00 00 00 07</code> ; <code><I → 07 00 00 00</code> .	Big-endian is the internet convention and removes any dependence on host CPU endianness — the same frame parses identically on x86 and ARM.
What does "wire" mean?	Slang for "the network". "Wire format" = the exact byte layout you agreed on.	A Python object is rich (<code>MsgType.ECHO</code>); the wire is dumb (<code>04</code>). encode/decode bridges them.
Do we hard-code sizes?	No — <code>HEADER_SIZE = struct.calcsize(HEADER_FORMAT)</code> . Add a field and the size follows by itself.	DRY. The same fact in two places will eventually disagree, and a stale header size silently breaks the parser.
What is the CRC field?	The last 4 bytes of the header. A fingerprint of the message; the receiver recomputes and compares.	Error detection for accidental corruption — bad cable, buggy framing. Not authentication.
Why <code>HEADER_SIZE - 4</code> ?	The crc is always the last 4 bytes, so it starts at <code>18-4 = 14</code> . A fingerprint cannot include itself.	The CRC covers header bytes 0–13 plus the payload. Covering the header matters — a flipped bit in <code>length</code> would otherwise go undetected.
Is <code>protocol.py</code> runnable?	Yes, but it printed nothing until we added a <code>__main__</code> block. Silence is not failure — a file of definitions has nothing to print.	<code>if __name__ == "__main__"</code> separates library code from script code.

Sockets and servers

I asked	Plain answer	Interview version
Who sets the header, and when?	Whoever is sending , inside <code>encode()</code> , once per message — not once per connection.	Handshake is per connection; framing is per message. Otherwise the receiver could not tell where message 2 begins.
Is a new connection made per message?	No. One connection carries thousands. The handshake costs a full round trip and <code>close()</code> parks the port in <code>TIME_WAIT</code> .	Same reason HTTP/1.1 introduced keep-alive and HTTP/2 added multiplexing.
Why not just <code>recv(29)</code> ?	The server does not know the message is 29 bytes until it reads the header. The only number it can trust blindly is <code>HEADER_SIZE</code> .	Read a fixed 18-byte header, learn <code>length</code> , then read exactly that many more.
What is wrong with <code>recv()</code> ?	It returns up to n bytes, not n. It may return 1. You must loop.	TCP is a stream — the kernel hands you whatever has arrived. Loop, requesting only the remainder each time.

Measurement

I asked	Plain answer	Interview version
What is throughput?	How many messages finish per second. Latency is how long one took. Different questions.	Latency is per request; throughput is aggregate. A queue can have great throughput and terrible latency at once.
What is a percentile?	Exam ranking. p95 = 95% of requests were faster than this number.	I report p50 for the typical case and p95/p99 for the tail, because the mean is skewed by outliers and describes no real request.
Why is the mean "not a real request"?	For 1, 1, 2, 2, 2, 3, 3, 4, 7, 20 the mean is 4.5 — and nobody waited 4.5 . Eight of ten were faster than the mean.	Averages hide tail latency. One 5-second stall among 1,000 requests barely moves the mean but ten real users waited 5 seconds.
Why "nearest-rank"?	<code>ceil(0.95 × n)</code> , then take that sample. You ask a real person rather than blending two of them.	Every number reported is an actually observed value, and there is no argument about which interpolation variant was used.
Why inject artificial loss?	A smoke alarm never beeps because there is never a fire — so you press TEST. Localhost never drops packets.	The recovery path is the whole feature; on loopback it would never execute. Chaos engineering, basically.
Why seed the random number generator?	Minecraft world seeds. Same seed = same drops every run.	An intermittent bug you cannot reproduce is a bug you cannot fix.

Debugging stories worth telling

SAY THIS OUT LOUD

"Any problems you had to debug?"

"Scapy crashed on Windows saying an interface was not found. I read the traceback and the bottom line said it broke while filling in the **source MAC address**. If you leave that blank, Scapy asks Windows for your default network card — and mine had picked a loopback adapter that does not exist without Npcap installed.

The quick fix was to type a MAC in myself. The better realisation was that I was writing packets to a **file**, not sending them — so I never needed a network card at all. The tool assumed I was transmitting and I had not noticed I picked up that assumption."

KEY POINT

Read the last line of a traceback first. It names what broke. Then ask: *do I actually need this thing at all?*

Part E · Interview question bank

Answer in **plain words first**, then name the technical term. If you lead with jargon and they push, you are stuck.

Protocol design

How do you frame messages over TCP?

- TCP is a byte stream with no message boundaries, so the application layer supplies them
- I used length-prefix framing: a fixed 18-byte header whose `length` field says how many payload bytes follow
- The alternative is delimiter framing, but that needs escaping inside the payload and makes the encoded size unpredictable
- Length-prefix passes arbitrary binary untouched and lets the receiver allocate exactly the right buffer

Why big-endian?

- Network byte order — the internet convention, so it interoperates
- It removes any dependence on host CPU endianness: the same frame parses identically on x86 and ARM
- The `!` also suppresses C-style alignment padding, so the wire layout is exactly what the format string says

Why a magic number and a version byte?

- Magic is a cheap sanity check — wrong first four bytes means the peer does not speak my protocol, or I have lost stream sync
- Better to close the connection than misinterpret bytes
- The version byte means a future incompatible format raises `UnsupportedVersionError` instead of being silently misparsed as v1

You already have TCP checksums — why CRC32?

- TCP's is 16-bit, weak, and only covers one hop's segment
- Mine covers my header fields and validates my own encode/decode path end to end
- And the UDP part of this project has no transport guarantee at all
- To be clear it is error **detection**, not authentication — an attacker can recompute it. For tamper resistance you want an HMAC

What is the danger of a length-prefixed protocol?

- Trusting the declared length
- It is a 4-byte unsigned int, so a hostile peer can claim 4 GB and make me allocate it from an 18-byte packet
- I cap payloads at 16 MiB and reject oversized frames **before** allocating anything

Why is the CRC checked last?

- It is the only check that touches every byte
- Malformed frames get rejected in $O(1)$ instead of $O(n)$
- Under a flood of garbage you do not want to hash all of it

Sockets

Implement a function that reads exactly N bytes from a socket.

- Loop while `len(buf) < n`, calling `recv(n - len(buf))`
- Treat `b""` as a closed connection and raise
- Accumulate in a `bytearray` — bytes concatenation is $O(n^2)$
- Special-case `n == 0`, otherwise `recv(0)` returns `b""` and looks like a disconnect

Why request `n - len(buf)` instead of `n`?

- Asking for more than you need can consume the start of the next message
- That desynchronises the stream permanently — every frame after it is garbage

`send` vs `sendall`?

- `send` may write fewer bytes than given and returns the count — you must loop
- `sendall` loops for you and raises on failure
- There is no `recvall`, which is why `recv_exact` has to exist

Why `SO_REUSEADDR`?

- After close, the port sits in `TIME_WAIT` for ~60 s so late packets from the old connection cannot be misdelivered
- Without the flag, restarting the server fails to bind
- `SO_REUSEPORT` is the different one — load balancing across processes

How do you stop one bad client killing the server?

- Per-connection `try/except ProtocolError` inside the worker
- `finally: conn.close()` guarantees no socket leak
- Errors are logged with the peer address and that connection is dropped — the accept loop never sees them

Concurrency

Threads vs asyncio — when do you pick which?

- Threads: fewer, longer-lived connections, or when you must call blocking libraries
- Asyncio: many concurrent connections that are mostly idle — thousands of sockets on one thread
- Threads cost ~8 MB of stack each and force kernel scheduling; tasks cost KBs and yield cooperatively
- I measured the crossover in this repo: threads won at 1 client, asyncio won from 10 up

Does the GIL make threads useless in Python?

- Only for CPU-bound work — one thread runs bytecode at a time
- This server is I/O-bound: the GIL is **released** during `recv`, `send` and `accept`
- So threads genuinely overlap here. The GIL would bite if I were compressing or encrypting payloads

Is `counter += 1` thread-safe in Python?

- **No**. It is load-add-store, three bytecodes, interruptible between them
- Two threads can interleave and lose an update
- I guard it with a `threading.Lock`
- In asyncio no lock is needed — a coroutine only suspends at an `await`, so code between two awaits is implicitly atomic

What is `drain()` for?

- `write()` buffers and returns immediately; it never blocks
- `drain()` suspends the task while the OS send buffer is full
- Without it, a slow reader makes the server buffer replies in memory until it dies — that is the backpressure bug

What is the danger with `asyncio`?

- One blocking call freezes **every** connection, because there is only one thread
- `time.sleep()`, a synchronous DB driver or heavy CPU work stalls the whole loop
- Fix: `await asyncio.sleep()`, async drivers, or `run_in_executor` for CPU work

Measurement

How do you benchmark a server properly?

- Reuse connections so you measure the protocol, not the handshake
- Synchronise the start with a barrier, or you never reach the concurrency you claim
- Discard warmup, use a monotonic clock, verify every response
- Report the distribution — p50/p95/p99 — not one average
- Suppress logging during the run; the observer must not change the measurement

Why p99 instead of average latency?

- The mean is skewed by outliers and describes no real request
- p99 is the experience of 1 user in 100 — and a page with 20 assets hits it about two thirds of the time
- I report p50 for the typical case and p95/p99 for the tail

What are the flaws in your benchmark?

- Client and server share one machine, so at 500 clients the load generator competes for CPU
- Loopback has no real network latency, loss or reordering
- Single run per configuration — no confidence intervals
- Fixing it means a separate load machine and repeated runs

Reliability

How do you make UDP reliable?

- Sequence numbers, acknowledgements and a retransmission timer — that is ARQ
- Sender blocks until the ACK for the current seq arrives, retransmitting on timeout
- Receiver delivers a seq once but **acknowledges it every time** it sees it
- Corrupt datagrams are not ACKed at all, so loss and corruption share one recovery path

Why must the receiver ACK duplicates?

- A duplicate exists **because** the previous ACK was lost
- Staying silent leaves the sender retrying forever — livelock
- Re-ACKing is the only way to break the loop

How does the sender distinguish a lost DATA from a lost ACK?

- **It cannot.** Both present as silence
- So it always retransmits, and correctness is pushed to the receiver's duplicate detection
- This is why "at-least-once delivery plus idempotent handling" is the standard distributed-systems pattern

What is wrong with stop-and-wait?

- Throughput is capped at one message per round trip regardless of bandwidth
- On a 200 ms link that is 5 messages/second no matter how fat the pipe
- Fix: sliding window, so N messages are in flight. Stop-and-wait is window size 1

Testing and tooling

What do you test in a protocol implementation?

- Round-trip equality across sizes, especially around the header boundary
- Every malformed input raising its **specific** exception
- The framing case that motivated the design: two frames in one write, split correctly
- Boundary values: empty payload, 1 MB payload, declared length above the cap

Your tests all pass. How do you know they are any good?

- Break something on purpose and check a test goes red — remove the CRC check and `test_bad_crc_*` must fail
- A test that never fails is not testing anything
- Next step is coverage or mutation testing to find the gaps

What is Scapy and why use it?

- A packet manipulation library — build, send, capture and dissect at any layer
- You control fields the OS normally fills in, so you can produce edge cases a normal socket cannot
- I used it to build my frames independently of my own encoder, then decode them with my own parser — a cross-check that both agree on the spec

What is the security angle?

- Raw packet crafting lets you spoof a source IP, forge headers or replay traffic
- Which is exactly why CRC32 is for corruption, not attackers
- If the threat model included an adversary I would add HMAC-SHA256 alongside the CRC

Questions to ask them

- What does your concurrency model look like — threads, async, processes, or a mix? What drove that choice?
- Do you have your own wire protocols, or is it mostly HTTP/gRPC?
- What do you watch in production — p50, p99, error budgets?
- How do you test failure paths? Do you inject faults?
- What is the most surprising performance problem the team has hit?

Part F · Cram sheet

One line per file

They ask about	You say
<code>protocol.py</code>	Length-prefix framing — TCP is a stream with no message boundaries, so the header says how many bytes follow.
the header	18 bytes: magic, version, type, seq, length, crc32. Big-endian, fixed size, so the receiver always knows how much to read first.
CRC32	Catches accidental corruption. Not security — an attacker can just recompute it.
<code>MAX_PAYLOAD_SIZE</code>	Length is a 4-byte int, so a peer can claim 4 GB. I cap it before allocating anything.
<code>recv_exact</code>	<code>recv</code> returns <i>up to</i> n bytes, not n. You have to loop or you misplay.
threaded vs async	Async won 47% on throughput at 500 clients; threaded won on p99. Different axes.
<code>udp_reliable.py</code>	Sequence numbers, ACKs and timeouts — rebuilding by hand what TCP gives you free.
<code>--loss</code>	Smoke alarm test button. Localhost never drops packets, so the recovery code never ran.
<code>tests/</code>	44 tests. Every malformed input must raise its <i>specific</i> exception, not just any error.
<code>craft.py</code>	Scapy builds the frame, my own parser decodes it. Two implementations, identical bytes.

Numbers to remember

Number	What it is
18	header size in bytes
14	<code>CRC_OFFSET</code> — where the crc field starts
4	bytes in the length field → max declarable size ~4.29 billion
16 MiB	our payload cap
44	tests
~40 ms	delay Nagle's algorithm can add per reply
~60 s	<code>TIME_WAIT</code> duration
~8 MB	virtual stack per OS thread
$0.95^{20} = 36\%$	chance all 20 page assets are fast

Words that make you sound senior

Term	Use it when talking about
length-prefix framing	the header
network byte order	the <code>!</code> in the struct format
single source of truth / DRY	<code>struct.calcsize</code> instead of hard-coding 18
make illegal states unrepresentable	why Message has only 3 fields
denial of service	the payload cap
head-of-line blocking	why async p99 was worse
Little's Law	$\text{latency} = \text{concurrency} \div \text{throughput}$
backpressure	<code>await writer.drain()</code>
C10K problem	the thread-per-client ceiling
chaos engineering	the <code>--loss</code> flag
at-least-once + idempotent	why duplicates must be handled
sliding window	the fix for stop-and-wait
<code>SO_REUSEPORT</code>	one event loop per core

The 60-second project pitch

SAY THIS OUT LOUD

"I built a binary application protocol from scratch — 18-byte header with magic, version, type, sequence number, length prefix and a CRC32 over both header and payload.

Then I implemented the server twice: thread-per-client and asyncio, and benchmarked them at 10, 100 and 500 concurrent connections. Async gave about 47% more throughput at 500, but roughly double the p99 latency — a single event loop serialises everything, while 500 OS threads let the kernel overlap syscalls across cores.

I also rebuilt reliable delivery on top of UDP by hand — stop-and-wait ARQ with sequence numbers, timeouts and duplicate suppression — with an artificial loss injector so the retransmission path actually gets exercised. And a Scapy tool that crafts packets carrying the protocol and decodes them with my own parser, which cross-checks both implementations against the spec."

Part G · Glossary

Term	Plain meaning	Real-life version
ACK	a message saying "I got it"	a delivery receipt
ARQ	Automatic Repeat reQuest — resend anything not acknowledged	re-posting a letter that never arrived
backpressure	slowing the sender down when the receiver cannot keep up	a checkout closing its lane when the belt is full
big-endian	biggest byte written first	writing the year as 2026, not 6202
byte	8 bits, a number 0–255	one character of text
bytes vs str	raw data vs text. Networks only carry bytes	a sealed parcel vs a description of it
C10K problem	the difficulty of handling 10,000 connections with threads	one waiter per table stops working at 10,000 tables
concurrency	many things in progress at once	juggling
CRC32	a 4-byte fingerprint of some data	writing the parcel weight on the box
daemon thread	a thread that does not stop the program from exiting	a shop assistant who leaves when the shop closes
deserialise / decode	bytes → object	unpacking a parcel
endianness	the order bytes of a number are written in	date format arguments
ephemeral port	a temporary port number the OS picks for a client	a cloakroom ticket number
event loop	one thread that switches between many paused tasks	one waiter, many tables
fixture (pytest)	reusable setup that cleans itself up	a finally: you write once
framing	marking where one message ends and the next begins	envelopes
GIL	Python's lock — one thread runs bytecode at a time	one microphone in a room
head-of-line blocking	one slow item delays everything behind it	a coupon argument at the till
HMAC	a checksum that needs a secret key — detects tampering	a wax seal only you can make
idempotent	doing it twice has the same effect as once	pressing a lift button again
latency	how long one request took	your personal wait
length-prefix	putting the size at the front of the message	"this letter is 300 characters"
Little's Law	wait = concurrency ÷ throughput	queue length divided by serving rate
loopback / 127.0.0.1	your own machine talking to itself	posting a letter to your own address
MAC address	the ID burned into a network card	a car number plate
magic number	fixed bytes that identify a format	a logo on the envelope
monotonic clock	a clock that only moves forward	a stopwatch, not a wall clock
Nagle's algorithm	TCP batching small writes together	waiting to fill the van before driving
percentile	position in a sorted list	exam ranking

promiscuous mode	a network card reading all traffic, not just its own	reading everyone's post
race condition	two threads interfering on shared data	two people editing the same line at once
REPL	the interactive <code>>>></code> Python prompt	a scratch pad
serialise / encode	object → bytes	packing a parcel
sliding window	allowing N unacknowledged messages in flight	several vans on the road at once
socket	one end of a network connection	one handset of a phone call
stop-and-wait	send one, wait for the ACK, then send the next	one van, one delivery, drive back
throughput	how many finish per second	customers served per hour
TIME_WAIT	a closed port kept reserved for ~60 s	a hotel room held after checkout
traceback	the list of steps before a crash	a receipt ending in what went wrong
venv	a private Python toolbox per project	your own toolbox, not the shared one
wire / wire format	the network / the exact byte layout on it	the road / the shape of the parcel

Where to go next

Improvement	What it teaches
One event loop per core with <code>SO_REUSEPORT</code>	how nginx and gunicorn actually scale
Sliding-window ARQ with selective ACK	how TCP really works
HMAC-SHA256 alongside the CRC	the difference between integrity and authenticity
A Wireshark Lua dissector	how protocol tooling is built
Property-based tests with Hypothesis	finding inputs you would never think of